

LUT2Graph

Intermediate Report — Stage 1: Topic Definition

Student: Olavo Barros **ID:** 102014

Course: INF 791 - Redes Complexas

DPI - Universidade Federal de Viçosa (UFV)

(a) General Topic

This project investigates **digital logic circuits** through the lens of complex-network theory. Specifically, we focus on circuits that have been synthesised by the Yosys open-source synthesis framework into networks of **Look-Up Tables (LUTs)** — the fundamental programmable logic elements of FPGAs. Each LUT implements an arbitrary Boolean function of up to k inputs by storing a truth table of 2^k binary entries.

The central idea is to map a synthesised netlist to a directed graph $G = (V, E)$ and then apply complex-network analysis to characterise, understand, and ultimately **prune** the circuit. Node and edge annotations derived from information-theoretic measures (Boolean Bias and Boolean Sensitivity) enrich the structural graph with functional meaning, enabling pruning strategies that are grounded in both topology and logic.

(b) Research Questions

The project is guided by the following research questions:

RQ1. Pruning via network metrics. Can nodes with extreme Boolean Bias ($\beta \approx 0$ or $\beta \approx 1$) and edges with low Boolean Sensitivity ($\sigma_j \approx 0$) be safely removed without compromising the logical correctness of the circuit? What is the trade-off between graph reduction ratio and circuit fidelity?

RQ2. Technique comparison. How do classical complex-network pruning techniques (threshold-based edge removal, backbone extraction, community-based reduction) compare with LUT-specific pruning guided by Boolean Sensitivity?

(c) Dataset and Network Construction

Dataset. The primary data source consists of synthesised netlists of benchmark circuits generated by TreeLUT [2] and Go-Fast [1]— a tool that optimizes XGBoost models to circuits for FPGA implementation. Was selected tree datasets, each containing 100 circuits of varying complexity (number of LUTs, depth, etc.), for the initial phase of the project. These circuits are representative of real-world applications and provide a rich testbed for evaluating pruning strategies.

Network construction. Each circuit is synthesised with **Yosys + ABC** using a 4-input LUT technology mapping (`abc -lut 4; write_json`). The resulting JSON netlist is then parsed by the `LUTGraphBuilder` class (Python / NetworkX) as follows:

- **Nodes.** Each LUT cell becomes a node annotated with its Boolean Bias $\beta \in [0, 1]$. Primary input and output ports are registered as boundary nodes.
- **Edges.** A directed edge (u, v) is added for every connection from a driver signal u to a driven LUT input v . The edge weight is the Boolean Sensitivity $\sigma_j \in [0, 1]$ of that specific LUT input, quantifying how much the output changes when that input is toggled.

References

- [1] Olavo Barros, Isabela Freitas, Pedro Coura, José Augusto M Nacif, Ricardo Ferreira, Adrián Alcolea, María Villarroya-Gaudó, and Javier Resano. Go fast: Gpu optimizer for fpga-lut approximate simulation and tuning. In *2025 XV Symposium on Computing Systems Engineering (SBESC)*, pages 1–6. IEEE, 2025.
- [2] Alireza Khataei and Kia Bazargan. Treelut: An efficient alternative to deep neural networks for inference acceleration using gradient boosted decision trees. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pages 14–24, 2025.